SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Name: ENTRIX — AI-Powered Integrated Enterprise Resource Planning Platform

Document Identifier: ENTRIX-SRS-EXHAUSTIVE-V3.0
Format Standard: IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018 (12-Section Comprehensive Framework)
Publication Status: Technical Baseline
Academic Affiliation: Ramdeobaba University, Nagpur — School of Computer Science & Engineering
Curriculum Session: 2026–27 | Semester: V | Programme: B.Tech. Computer Science & Engineering
Authors & Engineering Roles:

- Aditya Sakarkar (Roll No. A1-16) — System Architecture, DevOps & State Machines
- Chetan Gupta (Roll No. A1-50) — Backend Services, Database Schema & Security
- Pranav Rathi (Roll No. A1-52) — AI/ML Microservice, Pipeline Integration & Frontend

Table of Contents

1. Introduction
    - 1.1 Purpose of this Document
    - 1.2 Scope of the System
    - 1.3 System Overview & Context Diagram
    - 1.4 Definitions, Acronyms, and Abbreviations
    - 1.5 References
2. General Description
    - 2.1 User Objectives & Business Drivers
    - 2.2 User Personas, Characteristics & Permission Matrix
    - 2.3 General Capabilities & High-Level Benefits
    - 2.4 Operating Environment & Topology
    - 2.5 Assumptions, Dependencies & Constraints
3. Functional Requirements (Exhaustive Specification)
    - 3.1 Module 1: Identity, Authentication & Role-Based Access Control (SEC)
    - 3.2 Module 2: Human Resource Management (HRM)

    - 3.3 Module 3: Inventory Master & Multi-Warehouse Tracking (INV)
    - 3.4 Module 4: Multi-Stage Procurement Pipeline & State Machine (PROC)
    - 3.5 Module 5: Financial Accounting & Department Budget Enforcement (FIN)
    - 3.6 Module 6: Project Milestone & Task Allocation (PRJ)
    - 3.7 Module 7: AI/ML Decision Intelligence Microservice (AIML)
    - 3.8 Module 8: Immutable Audit Logging & Governance (AUD)
4. Interface Requirements
    - 4.1 User Interface (UI) Requirements
    - 4.2 Software Interfaces & Granular REST API Contracts
    - 4.3 Hardware & Network Interfaces
    - 4.4 Communication Protocols & Real-Time Event Streams
5. Performance Requirements
    - 5.1 Static Performance Constraints
    - 5.2 Dynamic Performance Constraints & Latency Budgets
6. Design & Implementation Constraints
7. Non-Functional Software Attributes
    - 7.1 Security Architecture & OWASP Mitigations
    - 7.2 Reliability, ACID Guarantees & Fault Tolerance
    - 7.3 Availability & Disaster Recovery
    - 7.4 Maintainability, Code Standards & Modularity
    - 7.5 Portability & Containerization Standards
8. Preliminary Schedule and Budget
    - 8.1 10-Week Engineering Work Breakdown Structure (WBS)
    - 8.2 Comprehensive Infrastructure Budget Estimate
9. Appendices & Technical Schemas
    - 9.1 Complete PostgreSQL Relational DDL (Data Definition Language)
    - 9.2 Mathematical Formulation of Machine Learning Algorithms
    - 9.3 Comprehensive State Transition Truth Table
    - 9.4 Detailed Use Case Scenarios
10. Uses of this SRS Document
11. Frequently Asked Questions (FAQs)
12. Conclusion & Technical Sign-Off

1. Introduction

1.1 Purpose of this Document

This Software Requirements Specification (SRS) provides an exhaustive, binding engineering description of the **ENTRIX Enterprise Resource Planning Platform**. It defines functional behaviors, inputs, outputs, database structures, mathematical algorithms, security guarantees, performance metrics, and validation criteria. It serves as the primary technical blueprint for software engineers, QA specialists, and university project review boards.

1.2 Scope of the System

ENTRIX is an integrated, cloud-native ERP platform built specifically to resolve data fragmentation, manual reconciliation, and lack of predictive decision support in small and medium-sized enterprises (SMEs).

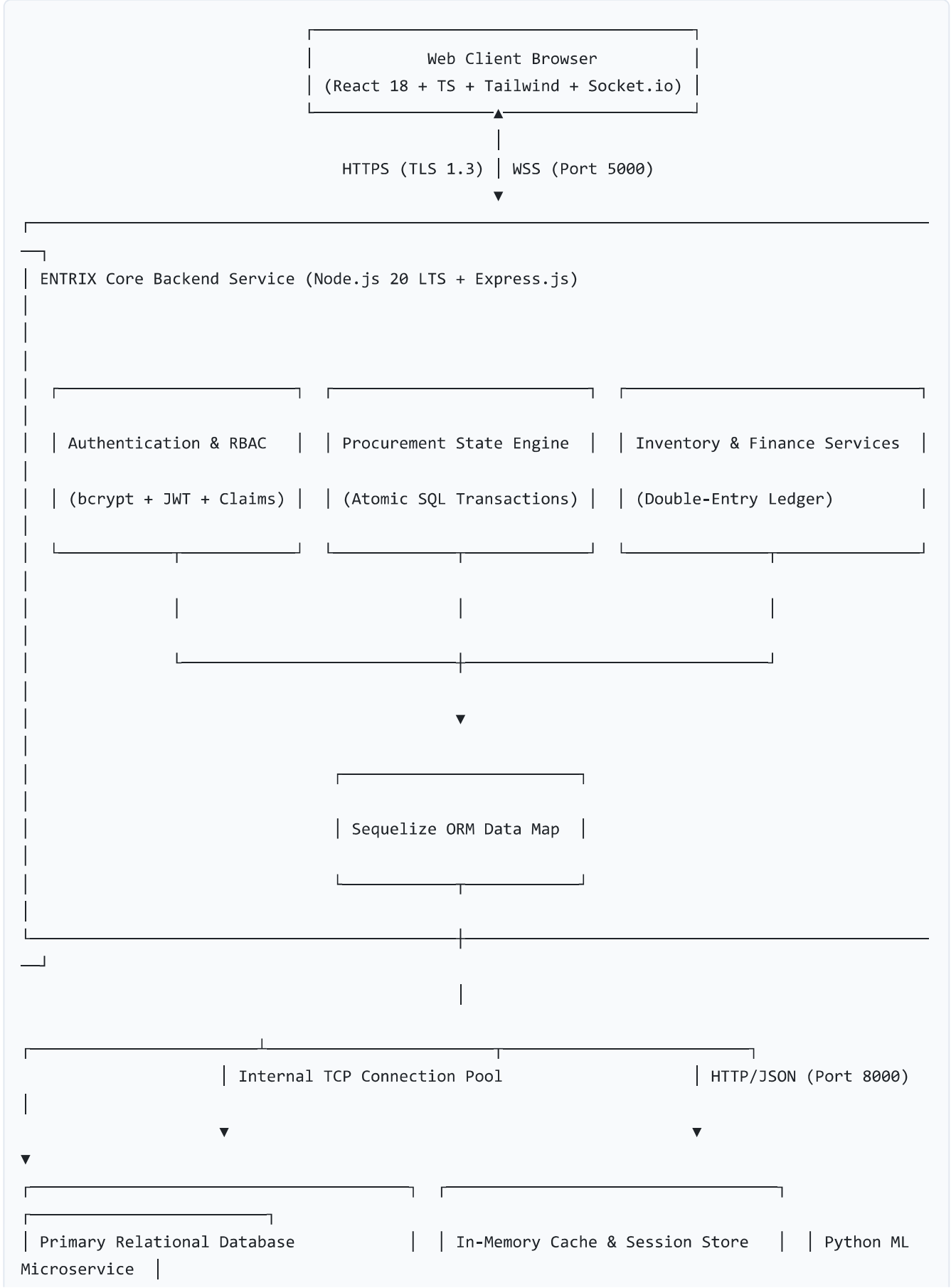
In-Scope Capabilities

- Centralized relational storage for personnel, ledger transactions, stock items, and vendor contracts.
- Deterministic 7-stage state machine for enterprise procurement requisitions.
- Automated 3-way invoice matching (Purchase Order ↔ Goods Received Note ↔ Vendor Invoice).
- Machine learning microservice providing 30-day SKU demand forecasts, expense outlier isolation, and project overrun predictions.
- Sub-second push notifications using WebSockets for approval dispatches.
- Cryptographically chained, append-only audit trail logging every administrative and transactional state alteration.

Out-of-Scope (Future Iterations)

- Native country-specific statutory tax compliance engines (e.g., automated Indian GST portal e-invoicing filing or European VAT MOSS integration).
- Hardware-level biometric clock-in hardware drivers (handled via REST web API ingestion instead).
- Multi-tenant database physical schema sharding (initial release uses single-tenant relational schema isolation).

1.3 System Overview & Context Diagram



```
| PostgreSQL 16.x Engine                 |  | Redis 7.x Server                   |  | FastAPI +
Scikit-Learn  |
| (ACID Tables, Foreign Keys, B-Trees)   |  | (JWT Blacklist, Metric Caching)    |  |
(RandomForest, IsoForest|
└────────────────────────────────────────┘  └────────────────────────────────────┘
└──────────────────────────┘
```

1.4 Definitions, Acronyms, and Abbreviations

- **A2A**: Agent-to-Agent protocol for machine communication.
- **ACID**: Atomicity, Consistency, Isolation, Durability.
- **B-Tree**: Balanced Tree indexing structure used in PostgreSQL indexes.
- **BOM**: Bill of Materials.
- **Bcrypt**: Adaptive cryptographic key derivation function based on the Blowfish cipher.
- **CORS**: Cross-Origin Resource Sharing.
- **CSP**: Content Security Policy.
- **DDL**: Data Definition Language.
- **ERP**: Enterprise Resource Planning.
- **GRN**: Goods Received Note (formal acknowledgement of physical warehouse delivery).
- **HMAC-SHA256**: Keyed-Hash Message Authentication Code using SHA-256.
- **Isolation Forest**: Unsupervised tree-based anomaly isolation algorithm.
- **JWT**: JSON Web Token (RFC 7519).
- **PO**: Purchase Order.
- **PR**: Purchase Requisition.
- **RBAC**: Role-Based Access Control.
- **SKU**: Stock Keeping Unit.
- **SPA**: Single Page Application.
- **WSS**: WebSocket Secure protocol over TLS.

1.5 References

1. IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
2. ISO/IEC/IEEE 29148:2018: *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*.
3. RFC 7519: *JSON Web Token (JWT)*.
4. RFC 8259: *The JavaScript Object Notation (JSON) Data Interchange Format*.
5. ENTRIX Project Synopsis: *Department of Computer Science & Engineering, Ramdeobaba University (2026)*.

2. General Description

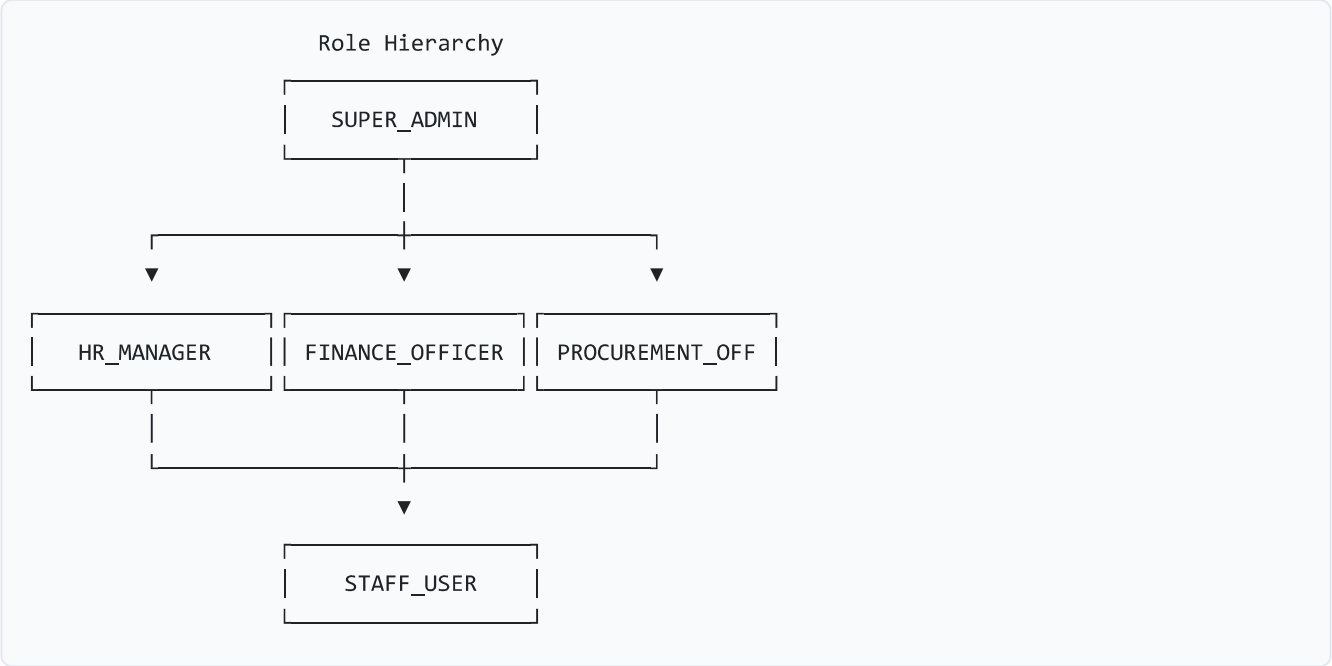
2.1 User Objectives & Business Drivers

In contemporary SME environments, organizational departments utilize disconnected SaaS platforms and localized Excel spreadsheets. This causes:

1. **Double Data Entry & Desynchronization**: Sales items sold are not instantly deducted from warehouse inventory.
2. **Approval Latency**: Procurement requests sit in email inboxes for days, delaying core business deliverables.
3. **Financial Fraud & Overspending**: Lack of automated anomaly checks permits unauthorized payments and undetected duplicate vendor invoices.
4. **Reactive Replenishment**: Inventory is only reordered when shelves are empty, leading to production downtime.

ENTRIX resolves these drivers by linking all transactional pipelines directly into an automated state machine backed by shared database storage and predictive analytics.

2.2 User Personas, Characteristics & Permission Matrix



Detailed Permission Matrix

Module / Operation	SUPER_ADMIN	HR_MANAGER	FINANCE_OFFICER	PROCUREMENT_OFF	STAFF_USER
User Provisioning & Roles	Full (CRUD)	None	None	None	None
Employee Directory	Full	Full (CRUD)	Read-Only	Read-Only	Read-Only (Self)
Attendance & Leave Approval	Full	Full	Read-Only	None	Submit Leave
Inventory Master & SKU Add	Full	None	Read-Only	Full (CRUD)	Read-Only
Stock Adjustments / Scrap	Full	None	Read-Only	Full (CRUD)	None
Submit Purchase Request (PR)	Full	Full	Full	Full	Full
Approve PR (< ₹10,000)	Full	Dept Lead Only	Dept Lead Only	None	None
Approve PR (>= ₹10,000)	Full	None	Mandatory Sign-off	None	None
Issue Purchase Order (PO)	Full	None	None	Full	None
Generate GRN (Goods Inward)	Full	None	None	Full	None
General Ledger & Invoicing	Full	None	Full (CRUD)	Read-Only	None
AI Demand Forecast Execution	Full	None	Read-Only	Full (Execute)	None
AI Expense Anomaly Triage	Full	None	Full (Clear/Flag)	None	None

Module / Operation	SUPER_ADMIN	HR_MANAGER	FINANCE_OFFICER	PROCUREMENT_OFF	STAFF_USER
Audit Log Inspection	Full	None	Read-Only	None	None

2.3 General Capabilities & High-Level Benefits

- **Automated Two-Tier Sign-Off**: Prevents unauthorized spending while ensuring routine purchases (< ₹10k) move instantly.
- **3-Way Matching Guarantee**: Prevents phantom payments by requiring PO, warehouse GRN, and vendor invoice data to align before payment authorization.
- **Predictive Restocking**: Replaces static min/max thresholds with dynamic 30-day machine learning demand curves.
- **Tamper-Evident History**: Any malicious change to finance or inventory records triggers a mismatch in audit hash verification.

2.4 Operating Environment & Topology

- **Client Application**: Cross-platform web browsers (Google Chrome 110+, Mozilla Firefox 115+, Microsoft Edge 110+, Safari 16+).
- **Backend Infrastructure**: Linux containers (Ubuntu 22.04 LTS base) deployed via Docker Engine v24.0+ and orchestrated via Docker Compose v2.20+.
- **Database Engine**: PostgreSQL 16.2 with write-ahead logging (WAL) enabled in replica mode.
- **In-Memory Store**: Redis 7.2 configured with AOF (Append-Only File) persistence.
- **Hosting Tier**: AWS t3.medium or containerized on-premise x86_64 virtualization.

2.5 Assumptions, Dependencies & Constraints

1. **Network Connectivity**: Internal LAN/WAN latency between application server and PostgreSQL database is <= 5 ms.
2. **System Time Synchronization**: All container hosts must run Network Time Protocol (NTP) to ensure millisecond-accurate audit timestamps.
3. **Browser Execution**: JavaScript execution must remain enabled on all client user agents.

3. Functional Requirements

3.1 Module 1: Identity, Authentication & Role-Based Access Control (SEC)

REQ-SEC-001: Secure User Authentication

- **Description**: The system must verify user identity via email and cryptographic password comparison.
- **Inputs**:
  - `email` (String, valid RFC 5322 format, lowercase, max 255 chars).
  - `password` (String, plain text, min 8 chars, at least 1 digit, 1 special symbol).
- **Processing**:
  1. System queries `users` table for unique email record. If missing, return `401 Unauthorized` with generic message.
  2. Compute `bcrypt.compare(password, password_hash)`.
  3. If invalid, increment `failed_login_attempts`. If attempts >= 5, lock account for 15 minutes.
  4. If valid, reset `failed_login_attempts` to 0. Generate stateless access JWT (15-min lifespan, signed using HMAC-SHA256) containing claims: `sub` (User UUID), `role` (Role string), `dept` (Department UUID), `exp` (UNIX timestamp).
  5. Generate cryptographically random refresh token (64 hex characters), store hashed refresh token in PostgreSQL with 7-day expiration.
- **Outputs**:
  - HTTP 200 OK with JSON payload: `{ "access_token": "...", "expires_in": 900, "user": { ... } }`.
  - Set-Cookie header with HTTP-only, Secure, SameSite=Strict `refresh_token`.

REQ-SEC-002: Token Refresh & Session Termination

- **Description**: Issue a new short-lived access token using a valid refresh token; revoke refresh token upon logout.
- **Inputs**: Incoming HTTP request containing refresh token cookie.
- **Processing**: Validate refresh token against database; confirm expiry time > current timestamp. Issue new access token. On logout, delete refresh token from database and publish access token identifier to Redis blacklist until expiration.
- **Outputs**: HTTP 200 OK with renewed access token or HTTP 204 No Content upon logout.

3.2 Module 2: Human Resource Management (HRM)

REQ-HRM-001: Employee Profile Lifecycle Management

- **Description**: Create, update, deactivate, and view employee organizational records.
- **Inputs**: `first_name`, `last_name`, `email`, `phone`, `date_of_birth`, `joining_date`, `department_id`, `designation`, `base_salary`, `bank_account_no`, `emergency_contact`.
- **Processing**: Verify `department_id` exists in `departments`. Insert row into `employees`. Generate unique employee code (`EMP-YYYY-XXXX`).
- **Outputs**: Created employee profile JSON with assigned UUID and HTTP 201 Created.

REQ-HRM-002: Daily Attendance & Leave Recording

- **Description**: Record daily presence and manage employee paid time off (PTO) balance.
- **Inputs**: `employee_id`, `date`, `status` (Enum: `PRESENT`, `ABSENT`, `ON_LEAVE`, `HALF_DAY`), `leave_type` (Enum: `SICK`, `CASUAL`, `UNPAID`).
- **Processing**: Prevent duplicate check-in records for the same employee on the same date via PostgreSQL compound unique index (`employee_id`, `date`). Deduct leave days from annual balance if `status == 'ON_LEAVE'`.
- **Outputs**: Attendance log confirmation and remaining leave balance count.

3.3 Module 3: Inventory Master & Multi-Warehouse Tracking (INV)

REQ-INV-001: SKU Master Catalog

- **Description**: Maintain an immutable material master catalogue.
- **Inputs**: `sku_code` (Unique String, uppercase alphanumeric, e.g. `RAW-STL-001`), `name`, `category`, `unit_of_measure` (e.g. `KG`, `PCS`, `LTR`), `unit_price`, `safety_stock_level`, `reorder_quantity`.
- **Processing**: Validate uniqueness of `sku_code`. Initialize stock count to zero across designated warehouse entries.
- **Outputs**: Material master record created in database.

REQ-INV-002: Real-Time Stock Increment / Decrement

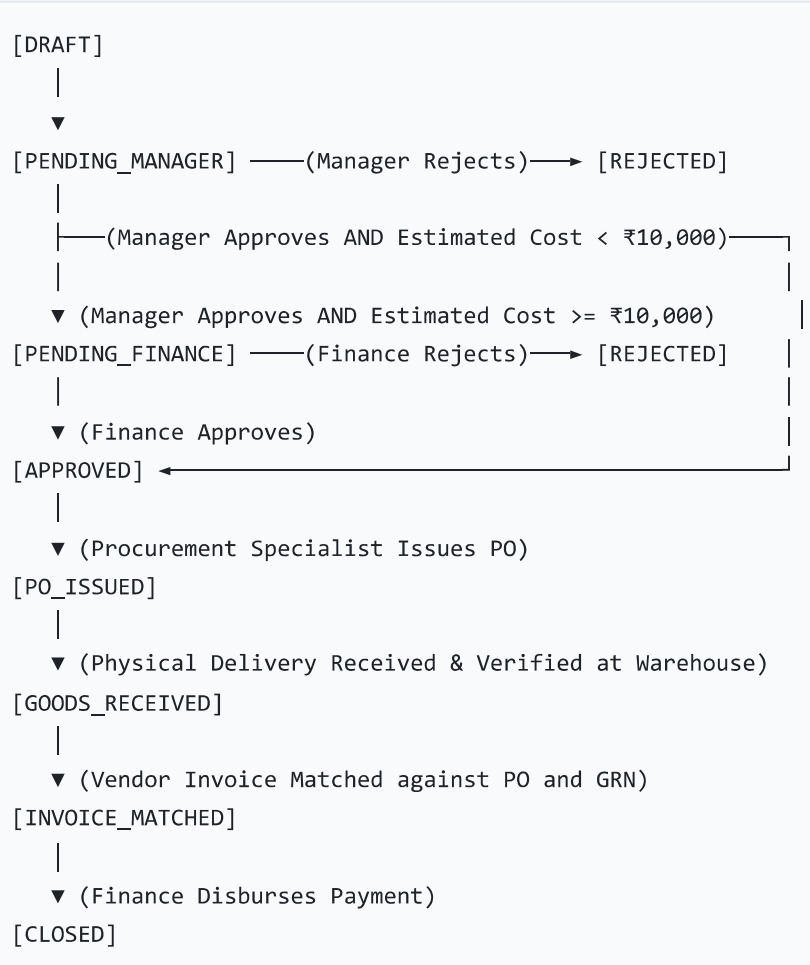
- **Description**: Mutate physical inventory counts strictly through traceable business transactions.
- **Processing Rules**:
  - **Stock Inward**: Can only occur via approved Goods Received Note (GRN) from procurement. Increments `stock_qty`.
  - **Stock Outward**: Can only occur via approved Project Work Order or Scrap Requisition. Decrements `stock_qty`.

    - **Safety Check**: If `stock_qty <= safety_stock_level`, execute asynchronous event `event_bus.publish('STOCK_LOW', { sku_id, current_qty })`.
- **Outputs**: Real-time stock balance update and Socket.io broadcast to procurement dashboards.

3.4 Module 4: Multi-Stage Procurement Pipeline & State Machine (PROC)

REQ-PROC-001: Deterministic Procurement State Transitions

The lifecycle of all enterprise procurement must follow the exact mathematical finite state machine:



REQ-PROC-002: Purchase Requisition (PR) Submission

- **Inputs**: `requester_id`, `department_id`, `item_id`, `quantity` (Integer > 0), `expected_delivery_date`, `justification_text`.
- **Processing**:
    1. Fetch `unit_price` from `inventory_items` where `id = item_id`.
    2. Compute `estimated_total = quantity * unit_price`.

3. Validate `estimated_total` against `department.remaining_quarterly_budget`. If budget exceeded, mark PR as `BUDGET_OVERRUN_FLAG`.
4. Insert record into `purchase_requests` with status `PENDING_MANAGER`.

- **Outputs**: Unique PR code (`PR-2026-XXXXX`), HTTP 201 Created, Socket.io push alert sent to relevant Department Manager.

REQ-PROC-003: Approval Routing Logic & Audit Capture

- **Inputs**: `pr_id`, `approver_id`, `action` (`APPROVE` or `REJECT`), `comments`.
- **Processing Rules**:
  - **Manager Step**: Must be executed by user where `role == 'HR_MANAGER' OR role == 'DEPT_LEAD'`.
    - If `action == 'REJECT'`, set status to `REJECTED`, record mandatory comment, and close workflow.
    - If `action == 'APPROVE' AND estimated_total < 10000.00`, set status directly to `APPROVED`.
    - If `action == 'APPROVE' AND estimated_total >= 10000.00`, set status to `PENDING_FINANCE`.
  - **Finance Step**: Must be executed by user where `role == 'FINANCE_OFFICER'`.
    - If `action == 'APPROVE'`, transition to `APPROVED`.
- **Outputs**: Updated PR status, notification dispatched to next stakeholder, and audit record written.

REQ-PROC-004: Goods Received Note (GRN) Generation & 3-Way Match

- **Inputs**: `po_id`, `received_quantity`, `warehouse_location_id`, `delivery_challan_no`.
- **Processing**:
  1. Open explicit PostgreSQL transaction (`BEGIN TRANSACTION`).
  2. Verify `received_quantity <= purchase_order.ordered_quantity`.
  3. Insert row into `goods_received_notes`.
  4. Execute `UPDATE inventory_items SET stock_qty = stock_qty + received_quantity WHERE id = item_id`.
  5. Check vendor invoice: ensure `abs(invoice.total - (po.unit_price * grn.received_quantity)) <= 0.01`.
  6. Transition state to `INVOICE_MATCHED`.
  7. Commit transaction (`COMMIT`).
- **Outputs**: Incremented inventory stock and updated invoice clearance record.

3.5 Module 5: Financial Accounting & Department Budget Enforcement (FIN)

REQ-FIN-001: Double-Entry General Ledger Postings

- **Description**: Maintain an immutable financial ledger with debits and credits.
- **Rule**: Every approved payment posting must balance: $\sum \text{Debits} = \sum \text{Credits}$∑Debits = ∑Credits.
- **Inputs**: `transaction_date`, `account_debit_id`, `account_credit_id`, `amount`, `reference_type` (`PO`, `PAYROLL`, `EXPENSE`), `reference_id`.
- **Processing**: Reject transaction if `amount <= 0.00`. Execute simultaneous debit and credit record creation in `ledger_entries`.

REQ-FIN-002: Department Budget Enforcement

- **Description**: Monitor and restrict departmental expenditure against allocated quarterly allowances.
- **Formulas**:

$$\text{Utilized Budget} = \sum \text{Actual Expenses} + \sum \text{Committed Approved PRs}$$

Utilized Budget = ∑Actual Expenses + ∑Committed Approved PRs

$$\text{Utilization Percentage} = \left(\frac{\text{Utilized Budget}}{\text{Allocated Quarterly Budget}}\right) \times 100$$

Utilization Percentage = (Allocated Quarterly BudgetUtilized Budget) × 100

- **Alert Levels**:
  - **80.0% to 99.9%**: Emit amber warning indicator on departmental dashboard.
  - **>= 100.0%**: Hard block automated PR approval. Requires executive manual override flag.

3.6 Module 6: Project Milestone & Task Allocation (PRJ)

REQ-PRJ-001: Project Work Breakdown & Task Delegation

- **Description**: Track operational projects, milestones, task assignments, and material consumption.
- **Inputs**: `project_name`, `client_id`, `start_date`, `deadline`, `budget_allocated`.
- **Processing**: Assign tasks to employees with estimated vs logged actual hours. Link inventory consumed by project tasks to material costs.

3.7 Module 7: AI/ML Decision Intelligence Microservice (AIML)

REQ-AIML-001: SKU Demand Forecasting Service

- **Algorithm**: `RandomForestRegressor` (Scikit-learn) with recursive multi-step forecasting.
- **Feature Set**:
  - `lag_7` (Consumption 7 days prior).

    - `lag_14` (Consumption 14 days prior).
    - `lag_30` (Consumption 30 days prior).
    - `rolling_mean_7` (7-day simple moving average).
    - `day_of_week` (Integer 0 to 6).
    - `month` (Integer 1 to 12).
- **Endpoint**: `POST /api/v1/ml/forecast/demand`
- **Payload Schema**:

```
json
{
  "sku_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "historical_days": 90,
  "forecast_horizon_days": 30
}
```

- **Response**: Array of 30 daily predicted consumption integers with 95% confidence intervals.
- **Fallback Protocol**: If Python service response time > 2.0s or fails, Node.js backend calculates 30-day simple moving average (SMA) locally:

$$\hat{y}_{t+1} = \frac{1}{30} \sum_{i=0}^{29} y_{t-i}$$

y^t+1 = 301 i=0∑29 yt−i

REQ-AIML-002: Expense Anomaly Detection Service

- **Algorithm**: `IsolationForest` (Scikit-learn), contamination rate set to 0.03 (3%).
- **Feature Vector**:

$$X = [\text{Normalized Amount}, \text{Vendor Frequency}, \text{Department Deviation Score}, \text{Hour of Day}]$$

X = [Normalized Amount, Vendor Frequency, Department Deviation Score, Hour of Day]

- **Endpoint**: `POST /api/v1/ml/anomaly/detect-expense`
- **Output**: Anomaly classification (`IS_ANOMALOUS` : boolean) and decision score ($s \in [-1, 1]$s ∈ [−1, 1]). Records with $s < -0.5$s < −0.5 are automatically flagged for senior finance audit review.

3.8 Module 8: Immutable Audit Logging & Governance (AUD)

REQ-AUD-001: Cryptographic Append-Only Audit Trail

- **Description**: Every state mutation on Users, Roles, Finance, and Procurement must generate a cryptographically chained audit log.
- **Hash Chaining Formula**:

$$\text{Hash}_i = \text{SHA256}(\text{Hash}_{i-1} \| \text{log_id} \| \text{actor_id} \| \text{action} \| \text{entity_id} \| \text{timestamp} \| \text{payload})$$

Hashi = SHA256(Hashi−1 ‖ log_id ‖ actor_id ‖ action ‖ entity_id ‖ timestamp ‖ payload)

- **Integrity Validation**: A scheduled nightly job traverses the audit log table, recalculating hashes. Any broken hash link triggers an immediate `SECURITY_INTEGRITY_VIOLATION` alert to the System Administrator.

4. Interface Requirements

4.1 User Interface (UI) Requirements

- **Design System**: Built using Tailwind CSS and Radix UI primitives.
- **Key Views**:
    1. **Executive Dashboard**: Real-time KPI counters (Monthly Spend, Pending Sign-offs, Warehouse Stock Alerts).
    2. **Procurement Pipeline Board**: Kanban view visualizing PRs moving across stages (`DRAFT` ➔ `PENDING` ➔ `APPROVED` ➔ `ORDERED` ➔ `RECEIVED`).
    3. **Inventory Data Grid**: Virtualized data table supporting 10,000+ rows with client-side column filtering, sorting, and CSV export.
    4. **AI Analytics Panel**: Visual charts built with Recharts rendering 30-day historical consumption curves juxtaposed against predicted demand lines.

4.2 Software Interfaces & Granular REST API Contracts

API Contract 1: User Login

- **Endpoint**: `POST /api/v1/auth/login`
- **Headers**: `Content-Type: application/json`
- **Request Body**:

```
json
{
  "email": "aditya.sakarkar@entrix.local",
  "password": "SecurePassword123!"
}
```

- **Responses**:
    - `200 OK` :

```
json
{
  "success": true,
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 900,
  "user": {
    "id": "c7a8b9d0-1234-5678-90ab-cdef12345678",
    "name": "Aditya Sakarkar",
    "email": "aditya.sakarkar@entrix.local",
    "role": "SUPER_ADMIN"
  }
}
```

- `401 Unauthorized`: `{ "success": false, "error": "Invalid email or password" }`
- `429 Too Many Requests`: `{ "success": false, "error": "Rate limit exceeded. Try again in 15 minutes." }`

API Contract 2: Submit Purchase Requisition

- **Endpoint**: `POST /api/v1/procurement/requests`
- **Headers**: `Authorization: Bearer <JWT>`, `Content-Type: application/json`
- **Request Body**:

```
json
{
  "item_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "quantity": 150,
  "required_by_date": "2026-11-15",
  "justification": "Restocking critical manufacturing components for Project Apollo"
}
```

- **Responses**:
  - `201 Created`:

```
json
{
  "success": true,
  "pr_id": "4e2c9b10-6789-4def-8123-abcdefabcdef",
  "pr_code": "PR-2026-00412",
  "estimated_total": 14250.00,
  "status": "PENDING_MANAGER",
  "requires_finance_approval": true
}
```

- `400 Bad Request`: `{ "success": false, "error": "Quantity must be greater than zero" }`
- `403 Forbidden`: `{ "success": false, "error": "User lacks PR creation privileges" }`

API Contract 3: Execute AI Demand Forecasting

- **Endpoint**: `POST /api/v1/ml/forecast/demand` (FastAPI Internal Endpoint)
- **Headers**: `X-Internal-Token: <SECRET_KEY>`, `Content-Type: application/json`
- **Request Body**:

```
json
{
  "sku_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "historical_series": [45, 52, 48, 60, 55, 53, 58, 62, 59, 64],
  "horizon_days": 30
}
```

- **Responses**:
  - `200 OK`:

```
json
{
  "sku_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "forecast": [63, 65, 62, 68, 70, 72, 69, 71, 74, 75],
  "model_metrics": { "mae": 3.42, "rmse": 4.18, "r2_score": 0.89 },
  "recommended_reorder_qty": 450
}
```

4.3 Hardware & Network Interfaces

- All internal network interfaces communicate over TCP/IP sockets on a dedicated Docker bridge network (`entrix_net`).

4.4 Communication Protocols & Real-Time Event Streams

- **REST APIs**: HTTP/1.1 and HTTP/2 over TLS 1.3.
- **WebSocket Protocol**: WSS utilizing Socket.io v4.7. Event channels:
  - `approvals:<user_id>` (Pushes pending requisition alerts).
  - `inventory:stock_alert` (Pushes low-stock warnings to warehouse leads).
  - `finance:anomaly_flag` (Pushes detected expense anomalies to finance team).

5. Performance Requirements

5.1 Static Performance Constraints

1. **Database Sizing**: Designed to store 10 years of audit history and 1,000,000 purchase records with disk usage <= 20 GB.
2. **Concurrent Connections**: PostgreSQL connection pool configured to maintain 20 persistent connections with automatic timeout at 10,000 ms.

5.2 Dynamic Performance Constraints & Latency Budgets

Metric	Target SLA	99th Percentile (p99)	Test Conditions
Authentication Verification	<= 50 ms	<= 100 ms	100 concurrent requests
Transactional CRUD APIs	<= 150 ms	<= 300 ms	200 concurrent requests
Complex Dashboard Aggregation	<= 350 ms	<= 600 ms	Queries scanning 100,000 rows
AI Demand Forecast Execution	<= 1,200 ms	<= 2,000 ms	Matrix history of 365 daily points
Real-time Push Notification	<= 250 ms	<= 500 ms	End-to-end Socket.io dispatch

6. Design & Implementation Constraints

1. **Pure Open-Source Stack (CON-001)**: The core platform must run exclusively on free, open-source technology components (Node.js, PostgreSQL, Redis, Python, React) to ensure zero operational licensing overhead for academic deployment.
2. **Stateless Business Logic Tier (CON-002)**: The Express.js application server must maintain no state in local memory. All session records must be maintained in Redis or PostgreSQL, permitting horizontal replication behind a reverse proxy.
3. **Database Referential Integrity (CON-003)**: Cascading deletes (`ON DELETE CASCADE`) are strictly forbidden on financial, inventory, and audit tables. All historical rows must be preserved or soft-deleted via `deleted_at IS NOT NULL`.

7. Non-Functional Software Attributes

7.1 Security Architecture & OWASP Mitigations

- **SQL Injection Prevention**: 100% of SQL queries executed through Sequelize ORM parameterized queries; raw string concatenation is forbidden.
- **Cross-Site Scripting (XSS)**: React engine automatically escapes HTML entities in JSX. Helmet.js sets `Content-Security-Policy: default-src 'self'`.
- **Brute Force Protection**: Express-rate-limit enforces maximum 5 login attempts per 15-minute window per IP.
- **Data at Rest Encryption**: Database tables containing sensitive payroll and banking numbers use PostgreSQL AES-256 `pgcrypto` functions.

7.2 Reliability, ACID Guarantees & Fault Tolerance

- All multi-table updates (e.g. GRN generation + Inventory Increment + Ledger Posting) run inside strict `READ COMMITTED` PostgreSQL transactions.
- In the event of an unhandled runtime error, the active transaction is rolled back completely, guaranteeing zero partial records.

7.3 Availability & Disaster Recovery

- System target operational uptime is 99.5% during business days.
- Automated database logical backups (`pg_dump`) execute every 24 hours at 02:00 UTC, archived with gzip compression.

7.4 Maintainability, Code Standards & Modularity

- Clean Architecture principles: Clear separation between Controllers, Services, Repositories, and Routes.
- Full TypeScript type-checking across frontend and backend with zero `any` allowance in production builds.

7.5 Portability & Containerization Standards

- Dockerfiles written following multi-stage build patterns, producing final minimal Alpine/Slim production images under 200 MB.

8. Preliminary Schedule and Budget

8.1 10-Week Engineering Work Breakdown Structure (WBS)

```
Week  1: [SRS Finalization & Relational Schema Modeling]
Week  2: [Docker Orchestration & Database Migrations]
Week  3: [JWT Auth, RBAC Middleware & User Management]
Week  4: [Inventory Master & Warehouse Stock Management]
Week  5: [Procurement Pipeline Engine & Approval Routing]
Week  6: [Goods Received Notes (GRN) & 3-Way Matching]
Week  7: [FastAPI ML Microservice & Demand Forecasting]
Week  8: [Isolation Forest Anomaly Detection Integration]
Week  9: [React SPA Dashboard & Socket.io Live Alerts]
Week 10: [End-to-End Testing, Security Hardening & Demo Setup]
```

8.2 Comprehensive Infrastructure Budget Estimate

Component	Target Provider	Academic / Prototype Tier	Production Scale Tier
Compute Application Tier	Render / Local Docker / AWS EC2	$0.00 (Free Tier / Student Credits)	$24.00 / month (t3.medium)
Relational Database	Supabase / Local PostgreSQL 16	$0.00 (Free Tier 500 MB)	$25.00 / month (Managed RDS)
In-Memory Cache (Redis)	Upstash / Local Redis 7	$0.00 (Free 10,000 commands/day)	$15.00 / month
Document Storage (Invoices)	AWS S3 Standard	$0.00 (5 GB Free Allowance)	$3.00 / month
Domain & SSL Certificates	Let's Encrypt / Cloudflare	$0.00 (Free Universal SSL)	$12.00 / year (Domain)
Total Net Cost	—	**$0.00 / month**	**~$67.00 / month**

9. Appendices & Technical Schemas

9.1 Complete PostgreSQL Relational DDL (Data Definition Language)

```sql
-- ENTRIX Relational Database Schema DDL
-- Target Engine: PostgreSQL 16.x

CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- 1. Departments Master
CREATE TABLE departments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    code VARCHAR(16) UNIQUE NOT NULL,
    name VARCHAR(128) NOT NULL,
    quarterly_budget NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Roles Master
CREATE TABLE roles (
    id SERIAL PRIMARY KEY,
    name VARCHAR(32) UNIQUE NOT NULL,
    permissions JSONB NOT NULL DEFAULT '{}'::jsonb
);

-- 3. Users Table
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(64) NOT NULL,
    last_name VARCHAR(64) NOT NULL,
    role_id INT NOT NULL REFERENCES roles(id),
    department_id UUID REFERENCES departments(id),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    failed_attempts INT NOT NULL DEFAULT 0,
    locked_until TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Material / Inventory Master
CREATE TABLE inventory_items (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    sku_code VARCHAR(32) UNIQUE NOT NULL,
    name VARCHAR(128) NOT NULL,
    category VARCHAR(64) NOT NULL,
    unit_price NUMERIC(10, 2) NOT NULL CHECK (unit_price >= 0.00),
    stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    safety_stock INT NOT NULL DEFAULT 10,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5. Purchase Requisitions (PR)
CREATE TYPE pr_status_enum AS ENUM (
    'DRAFT', 'PENDING_MANAGER', 'PENDING_FINANCE',
    'APPROVED', 'PO_ISSUED', 'GOODS_RECEIVED', 'INVOICE_MATCHED', 'CLOSED', 'REJECTED'
);

CREATE TABLE purchase_requests (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    pr_code VARCHAR(32) UNIQUE NOT NULL,
    requester_id UUID NOT NULL REFERENCES users(id),
    department_id UUID NOT NULL REFERENCES departments(id),
    item_id UUID NOT NULL REFERENCES inventory_items(id),
    quantity INT NOT NULL CHECK (quantity > 0),
    estimated_total NUMERIC(12, 2) NOT NULL,
    status pr_status_enum NOT NULL DEFAULT 'DRAFT',
    rejection_reason TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 6. Purchase Orders (PO)
CREATE TABLE purchase_orders (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    po_number VARCHAR(32) UNIQUE NOT NULL,
    pr_id UUID UNIQUE NOT NULL REFERENCES purchase_requests(id),
    vendor_name VARCHAR(128) NOT NULL,
    total_amount NUMERIC(12, 2) NOT NULL,
    issued_date DATE NOT NULL DEFAULT CURRENT_DATE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 7. Goods Received Notes (GRN)
CREATE TABLE goods_received_notes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    grn_code VARCHAR(32) UNIQUE NOT NULL,
    po_id UUID NOT NULL REFERENCES purchase_orders(id),
    received_quantity INT NOT NULL CHECK (received_quantity > 0),
    received_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    verified_by UUID NOT NULL REFERENCES users(id)
```

```
);

-- 8. General Ledger Postings
CREATE TABLE ledger_entries (
    id BIGSERIAL PRIMARY KEY,
    transaction_uuid UUID DEFAULT uuid_generate_v4(),
    account_code VARCHAR(32) NOT NULL,
    debit_amount NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
    credit_amount NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
    reference_id UUID,
    posted_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 9. Cryptographically Chained Audit Logs
CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,
    actor_id UUID REFERENCES users(id),
    action VARCHAR(64) NOT NULL,
    entity_name VARCHAR(64) NOT NULL,
    entity_id UUID NOT NULL,
    diff_payload JSONB NOT NULL,
    previous_hash VARCHAR(64) NOT NULL,
    current_hash VARCHAR(64) NOT NULL,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- B-Tree Performance Indexes
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_inventory_sku ON inventory_items(sku_code);
CREATE INDEX idx_pr_status ON purchase_requests(status);
CREATE INDEX idx_pr_dept ON purchase_requests(department_id);
CREATE INDEX idx_audit_entity ON audit_logs(entity_name, entity_id);
```

9.2 Mathematical Formulation of Machine Learning Algorithms

Algorithm 1: Demand Regression Model (Random Forest)

The future stock demand $\hat{y}$y^ is estimated as the aggregate ensemble mean of BB independent regression decision trees T_b Tb:

$$\hat{y}(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x)$$

y^(x) = B1 b=1∑B Tb(x) Each split in tree T_b Tb is chosen to minimize Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \bar{y})^2$$

MSE = N1 i=1∑N (yi − y¯)2

Algorithm 2: Unsupervised Anomaly Isolation (Isolation Forest)

Given a dataset of nn financial invoices, the anomaly score $s(x, n)$s(x, n) for an invoice instance xx is defined as:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

s(x, n) = 2−c(n)E(h(x))Where:

- $h(x)$h(x) is the path length (number of splits required to isolate point xx in an isolation tree).
- $E(h(x))$E(h(x)) is the expectation of $h(x)$h(x) across all ensemble trees.
- $c(n)$c(n) is the average path length of unsuccessful search in a Binary Search Tree (BST):
  $$c(n) = 2\ln(n-1) + \gamma - \frac{2(n-1)}{n}$$
  c(n) = 2 ln(n − 1) + γ − n2(n − 1)where $\gamma \approx 0.5772156649$γ ≈ 0.5772156649 (Euler-Mascheroni constant).

Decision Criteria:

- If $s(x, n) \to 1.0$s(x, n) → 1.0 (or raw score < -0.5< −0.5), the transaction is flagged as **ANOMALOUS FRAUD CANDIDATE**.
- If $s(x, n) < 0.5$s(x, n) < 0.5, the transaction is considered standard operational expenditure.

9.3 Comprehensive State Transition Truth Table

Current State	Event / Trigger	Actor Permitted	Guard Condition	Next State	Database Action
`DRAFT`	`SUBMIT_PR`	Standard Employee	Item exists, Qty > 0	`PENDING_MANAGER`	Insert PR record
`PENDING_MANAGER`	`REJECT`	Dept Manager	Rejection comment provided	`REJECTED`	Update status, notify requester
`PENDING_MANAGER`	`APPROVE`	Dept Manager	Estimated Cost < ₹10,000	`APPROVED`	Update status, notify procurement
`PENDING_MANAGER`	`APPROVE`	Dept Manager	Estimated Cost >= ₹10,000	`PENDING_FINANCE`	Update status, notify finance
`PENDING_FINANCE`	`REJECT`	Finance Officer	Rejection comment provided	`REJECTED`	Update status, notify requester
`PENDING_FINANCE`	`APPROVE`	Finance Officer	Dept Budget sufficient	`APPROVED`	Commit budget reservation
`APPROVED`	`GENERATE_PO`	Procurement Lead	Registered vendor selected	`PO_ISSUED`	Insert PO record
`PO_ISSUED`	`RECEIVE_GOODS`	Warehouse Officer	Physical shipment verified	`GOODS_RECEIVED`	Insert GRN, increment inventory
`GOODS_RECEIVED`	`VERIFY_INVOICE`	Finance Officer	3-way tolerance <= 0.01	`INVOICE_MATCHED`	Match ledger records
`INVOICE_MATCHED`	`DISBURSE_PAY`	Finance Officer	Payment gateway confirmation	`CLOSED`	Post double-entry ledger debit/credit

9.4 Detailed Use Case Scenarios

Use Case UC-01: End-to-End Procurement Lifecycle

- **Primary Actor**: Employee (Requester), Department Manager, Finance Manager, Procurement Specialist, Warehouse Officer.
- **Preconditions**: User is authenticated; item SKU exists in catalogue.
- **Main Flow**:
    1. Employee submits PR for 20 units of `RAW-STL-001` (Estimated total: ₹24,000).
    2. System detects amount >= ₹10,000; sets status to `PENDING_MANAGER`.
    3. Department Manager logs in, reviews justification, and clicks **Approve**.
    4. State advances to `PENDING_FINANCE`; Finance Officer receives push notification.

5. Finance Officer validates quarterly budget and clicks **Approve**. State advances to `APPROVED`.
6. Procurement Specialist selects approved supplier and clicks **Generate PO**.
7. Supplier delivers material. Warehouse Officer counts items and submits GRN for 20 units.
8. System atomically updates `stock_qty = stock_qty + 20` and generates audit record.
9. Finance matches invoice and clicks **Authorize Payment**. Status transitions to `CLOSED`.

- **Postconditions**: Stock is updated in inventory master; ledger accounts balanced; audit log entry verified.

10. Uses of this SRS Document

1. **For Core Engineering Team**: Acts as the single technical source of truth for database DDL schemas, HTTP endpoint contracts, and state transition logic.
2. **For Software Quality Assurance (QA)**: Dictates boundary condition test suites, state machine validation matrices, and load performance testing baselines.
3. **For Maintenance & Support Engineers**: Outlines failure modes, error response codes, and microservice recovery procedures.
4. **For Academic Evaluators**: Proves comprehensive engineering design rigor, architectural competence, and systematic requirement specification according to IEEE/ISO guidelines.

11. Frequently Asked Questions (FAQs)

Q1: Why are monetary values stored as `NUMERIC(12, 2)` instead of `FLOAT` or `DOUBLE`?
Answer: Floating-point representations suffer from binary rounding errors (e.g., $0.1 + 0.2 = 0.30000000000000004$0.1 + 0.2 = 0.30000000000000004). In financial and ERP accounting systems, arbitrary-precision decimal numbers (`NUMERIC`) are mandatory to ensure that penny balances never disappear during division or summation.

Q2: What happens if the Python ML microservice crashes during production hours?
Answer: The Node.js application server wraps all internal calls to the FastAPI microservice inside an asynchronous circuit breaker (`try/catch` with a 2,000 ms timeout). If the microservice is unreachable, the system automatically falls back to calculating historical moving averages without impeding core procurement or transactional workflows.

Q3: How does hash chaining prevent database tampering by database administrators?
Answer: Each audit log entry includes a SHA-256 hash that incorporates the hash of the immediately preceding record. If an actor directly modifies a historical database row via SQL CLI, all subsequent hash values in the chain become mathematically invalid, instantly surfacing during the automated nightly verification sweep.

12. Conclusion & Technical Sign-Off

The **ENTRIX Enterprise Resource Planning Platform** represents a modern, resilient, and intelligent solution to enterprise data fragmentation. By formalizing this exhaustive Software Requirements Specification, the development team has established an unambiguous technical contract governing every data entity, state change, and algorithmic model. Strict adherence to this document guarantees delivery of an academically superior and commercially viable software platform.